

Create a Spring Web Project using Maven

After Downloading the **spring-learn** file from the website, open it in any IDE and first open the terminal inside that file and run the following command:

mvn clean package

Output:

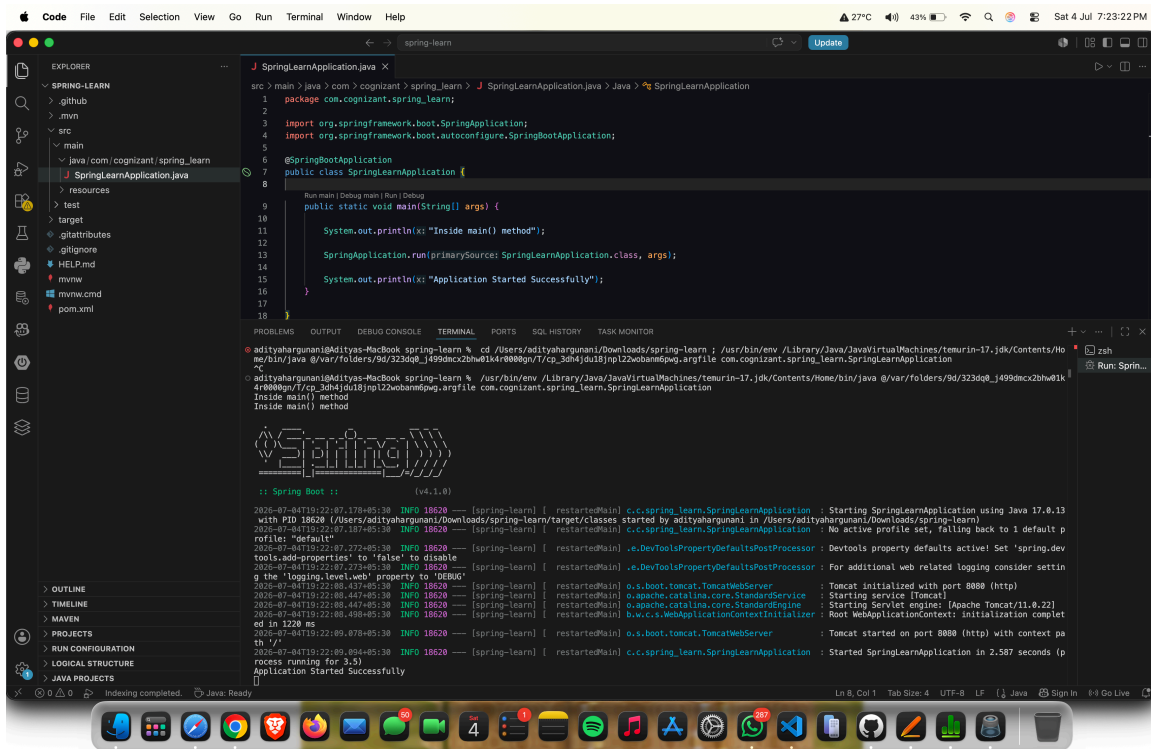
The screenshot shows an IDE window with the following components:

- Top Bar:** Code, File, Edit, Selection, View, Go, Run, Terminal, Window, Help. System status: 27°C, 47% battery, signal strength, and date/time: Sat 4 Jul 2023 00:07 PM.
- Left Sidebar:** Explorer view showing the project structure: `SPRING-LEARN` > `resources` > `test` > `SpringLearnApplicationTests.java`.
- Main Editor:** Displays the `SpringLearnApplicationTests.java` file. The code includes imports for `org.springframework.test.context.junit4.SpringRunner` and `org.springframework.test.context.support.AnnotationConfigContextLoaderUtils`. The `@SpringRunner` annotation is used on the `SpringLearnApplicationTests` class.
- Bottom Panel:** The `Run` tab is active, showing the execution output. The output includes the Spring logo, version information (2.7.4), and the successful execution of the `SpringLearnApplicationTests` class. The output also shows the application's configuration and the successful execution of the `SpringLearnApplicationTests` class.

then run the **SpringLearnApplication.java** file using run option or write in terminal:

```
mvn spring-boot:run
```

We will get the output as:



Explanations:

1. src/main/java

Contains the application source code such as controllers, services, models, repositories, and the main application class.

2. src/main/resources

Stores application configuration files like application.properties/application.yml, static resources, and templates.

3. src/test/java

Contains JUnit test classes used to test the application.

4. SpringLearnApplication.java

This is the entry point of the Spring Boot application. The main() method calls SpringApplication.run(), which starts the Spring application context and the embedded web server.

5. Purpose of @SpringBootApplication

This annotation combines:

- @Configuration: Marks the class as a source of bean definitions.

- @EnableAutoConfiguration: Automatically configures Spring Boot based on dependencies.
- @ComponentScan: Scans the package for Spring components.

6. pom.xml

The pom.xml file is the Maven Project Object Model. It defines project information, Java version, dependencies, plugins, and build configuration.

Important sections:

- groupId: com.cognizant
- artifactId: spring-learn
- version: Project version
- parent: spring-boot-starter-parent
- dependencies: spring-boot-starter-web, spring-boot-devtools, spring-boot-starter-test
- build plugins: Used for compiling and packaging the application.

7. Dependency Hierarchy

The dependency hierarchy displays all direct and transitive Maven dependencies. In VS Code, open the Maven panel or execute:
mvn dependency:tree

8. Running the Project

Build:

mvn clean package

Run:

mvn spring-boot:run

9. Expected Console Output

Inside main() method

Started SpringLearnApplication

Application Started Successfully